

Tecnológico de Monterrey

Campus Puebla

Course

Movilidad Inteligente (MR3004C.601)

Assignment

M4 Activity 2: Implementing a Pure Pursuit Controller in ROS 2

Professor

Arturo Daniel Sosa Cerón

Team

Rebeca Saldaña Espinoza A01735498
José Alfredo Guerrero Fernández A01736556
Jose David Bautista Zuñiga A01734266
Sina Jovy A01833561
Gabriel Said Vera Carballo A00839097
Luis Héctor Muñoz Del Razo A01738595

GitHub repository

https://github.com/Gabrysaid/pure_pursuit_ws

Date

September 27, 2026

1. Implementation

We created the `ament_python` package `pure_pursuit_controller` with two nodes and a ROS free module (`pure_pursuit_core.py`) that holds all the math, so it can be unit tested without a simulator (6 `pytest` cases: yaw conversion, straight line, turn direction, bicycle mapping and index monotonicity over two laps). **path_recorder** subscribes to `/odom` (`nav_msgs/Odometry`) and appends (x, y) only when the Euclidean displacement from the last stored point exceeds `min_distance = 0.5` m. The list is written to CSV from a `finally` block and from a context `on_shutdown` hook, so `Ctrl+C` always produces the file. The same node, launched with another output name, logs the executed path during autonomous runs.

pure_pursuit_node keeps clean callback boundaries: the odometry callback only stores the state (x, y, θ , v) and a timestamp, while a 20 Hz timer runs the controller and publishes `/cmd_vel` (`geometry_msgs/Twist`). This decouples the command rate from the odometry rate. If odometry is older than 0.5 s the node commands zero velocity, it stops by itself after one lap, and on `Ctrl+C` it sends a final stop because we disable the default `rcipy` signal handler. All gains are ROS parameters loaded from `config/pure_pursuit.yaml` and can be changed live through a parameter callback (`ros2 param set /pure_pursuit_node lookahead_gain 0.8`). For debugging, the node also publishes the target point and the signed cross track error. `launch/pure_pursuit.launch.py` includes `prius_bringup/gz_sim.launch.py` and, after a short delay, starts the controller and the trajectory logger. Command-line arguments set the speed, look-ahead and output file name. The reference was recorded with `teleop_twist_keyboard` using `record_path.launch.py`: 374 waypoints (192 m).

2. Control formulation

The planar heading is extracted from the odometry quaternion with the ZYX convention:

$$\theta = \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2))$$

The look-ahead distance grows with speed, $L_d = L_{d,\min} + k \cdot v$, clipped to 15 m. To avoid index jumps, the closest waypoint is searched only forward from the previous one inside a 60 point window, using an unwrapped counter so the closed loop is crossed without jumping back to index 0. Starting there, we advance until the first waypoint at a distance $\geq L_d$ from the car; that is the target (x_t, y_t). With $\alpha = \text{atan2}(y_t - y, x_t - x) - \theta$ (wrapped to $(-\pi, \pi]$), the arc through the car and the target has curvature $\kappa = 2 \sin \alpha / L_d$. For a bicycle model with wheelbase L, $\tan \delta = L \kappa$, which gives the steering law and, since $d\theta/dt = v \kappa$, the yaw rate sent as `angular.z`:

$$\delta = \tan^{-1}(2L \sin \alpha / L_d), \quad |\delta| \leq 0.5 \text{ rad}$$

$$d\theta/dt = v \tan \delta / L$$

$L = 2.7$ m and the 0.5 rad limit are taken from the `<wheel_base>` and `<steering_limit>` tags of the Prius AckermannSteering plugin. The plugin inverts (3) with the same L, so the commanded δ is exactly what reaches the front wheels. The commanded speed is used in (3) and the measured speed in L_d . Defaults: $v = 4$ m/s, $L_{d,\min} = 3$ m, $k = 0.4$ s.

3. Tracking performance

The cross track error (CTE) is the distance from every logged point to the closest segment of the reference polyline (scripts/analyze_tracking.py). Reference points whose curvature, computed over a 10 m stencil to filter the teleoperation wobble, exceeds 0.02 m^{-1} ($R < 50 \text{ m}$) are labeled curve; the rest are straights. All numbers come from Gazebo runs (Gazebo Sim 10, Prius model, $v = 4 \text{ m/s}$) against the recorded reference, 374 waypoints (192 m). The teleoperated drive was trimmed where it rejoins the start straight (0.19 m gap), so the loop closes with a sharp junction.

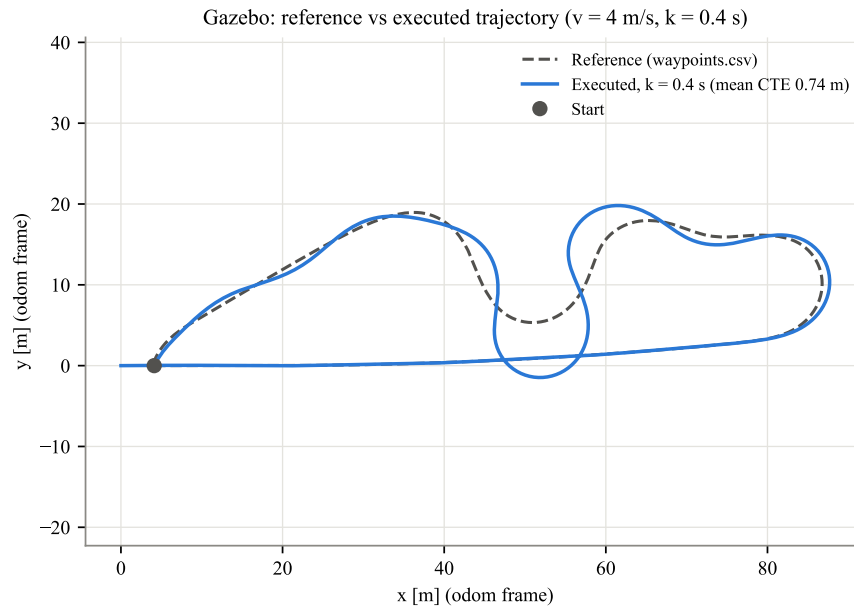


Fig. 1. Reference path (waypoints.csv) against the executed trajectory (actual_trajectory_main.csv) in the odometry frame, equal axis scale.

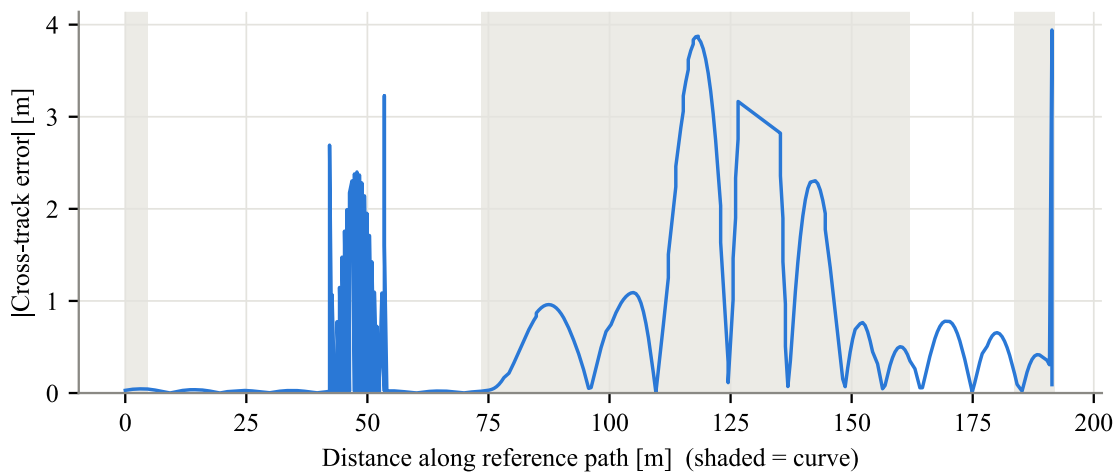


Fig. 2. Absolute CTE along the reference path ($k = 0.4 \text{ s}$); shaded regions are curves. The spikes near 45 m are points of the S-bend excursion that lie closest to the start straight.

For the default gains the mean CTE over the whole lap is 0.74 m (RMS 1.22 m, maximum 3.94 m). On straights the mean is 0.36 m and on curves 1.11 m, 3.1 times larger. Two effects explain the difference. First, corner cutting: the target point is L_d ahead, so it enters the curve before the car and the controller turns early, following a chord inside the arc. Second, steering lag: when we

commanded $\omega = 0.3$ rad/s in Gazebo the odometry reported only 0.275 rad/s after the transient, because the steering joints need time to reach the commanded angle. In the tight S-bend ($R \approx 6$ m, close to the minimum radius $L/\tan 0.5 = 4.9$ m) the car saturates the steering, overshoots and loops once before rejoining the path, which produces the maximum error.

4. Look-ahead tuning

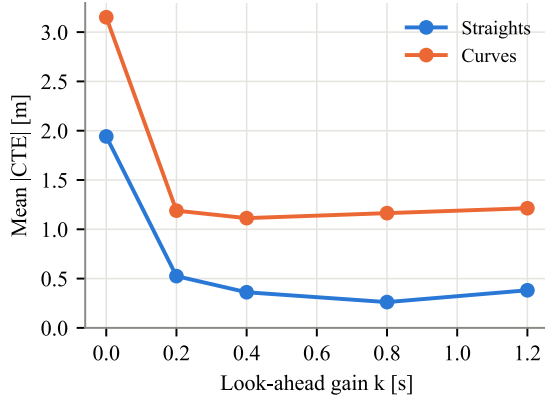


Fig. 3. Mean CTE versus look-ahead gain k ($L_{d,\min} = 3$ m, $v = 4$ m/s), Gazebo.

Run	Mean	RMS	P95	Max	Straight	Curve
k = 0.0	2.57	3.79	8.44	10.49	1.94	3.15
k = 0.2	0.86	1.28	2.87	3.94	0.52	1.19
k = 0.4	0.74	1.22	3.05	3.94	0.36	1.11
k = 0.8	0.72	1.18	2.90	3.97	0.26	1.16
k = 1.2	0.82	1.30	2.95	4.21	0.38	1.21

Table 1. CTE in metres, one Gazebo run per k , simulation restarted before each run.

The sweep shows the expected U shape. With a short look-ahead ($k = 0$, $L_d = 3$ m) the geometric gain $2L/L_d^2$ is high and, combined with the steering lag, the car overshoots every corner and loops around the path: the mean CTE is 2.57 m and the maximum 10.49 m. That run also stopped before closing the lap, because the forward index search advanced while the car was looping. Increasing k damps the response: at $k = 0.2$ s the mean drops to 0.86 m. Between $k = 0.4$ and 0.8 s the curve is flat (0.74 and 0.72 m); the longer preview keeps improving the straights (0.36 to 0.26 m) but the curve error already grows (1.11 to 1.16 m). At $k = 1.2$ s the chord to the target cuts the inside of the corners, the curve error rises to 1.21 m and the mean to 0.82 m. Making L_d proportional to speed keeps the preview time constant, so the same k transfers across speeds. The lowest mean CTE was at $k = 0.8$ s, but its difference with $k = 0.4$ s (0.02 m) is smaller than what a single run can resolve. We selected $k = 0.4$ s with $L_{d,\min} = 3$ m because it gives the lowest curve error and the lowest maximum error, i.e. the best compromise between oscillation and corner cutting.

Conclusion. The package builds with colcon without warnings, runs the controller on a fixed-rate timer and reproduces the recorded path with a mean error of 0.74 m at 4 m/s. The main limitation is the tight S-bend, where the steering limit and lag dominate. A curvature-dependent speed profile or a Stanley term for the lateral error would reduce it.

References

- [1] R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm," Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, 1992.
- [2] J. M. Snider, "Automatic Steering Methods for Autonomous Automobile Path Tracking," Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-09-08, 2009.
- [3] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York: Springer, 2012.
- [4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.
- [5] Open Robotics, "ROS 2 Lyrical Luth documentation," docs.ros.org/en/lyrical, 2026.
- [6] Open Robotics, "Gazebo Sim AckermannSteering system," gazebo.org/api/sim/10, 2026.
- [7] A. D. Sosa Cerón, "movilidad_inteligente" course repository (prius_bringup), github.com/dsosa114/movilidad_inteligente, 2026.

Appendix: use of AI

Claude (Anthropic) was used as a programming assistant to draft the package code, the analysis scripts and a first version of this text. The team reviewed, tested and edited all of it, ran the simulations and is responsible for the results.